



**WYŻSZA SZKOŁA
INFORMATYKI I ZARZĄDZANIA**
z siedzibą w Rzeszowie

Sprawozdanie z realizacji projektu

System monitorowania jakości nawierzchni drogowej w czasie rzeczywistym

PROJEKT ZESPOŁOWY

Prowadzący:

mgr inż. Aleksander Mysakowec

Wykonawcy:

Roman Bezshchasnyi, Grupa P1, 69930

Rostysław Yaremko, Grupa P1, 69958

Rzeszów 2026

Spis treści

1	Wprowadzenie	3
1.1	Cel i geneza projektu	3
1.2	Cel dokumentu	3
1.3	Zakres i założenia	3
2	Zespół projektowy i organizacja pracy	3
2.1	Skład zespołu i podział obowiązków	4
2.2	Macierz odpowiedzialności RACI	4
2.3	Narzędzia współpracy	5
3	Zarządzanie projektem	5
3.1	Karta projektu	6
3.2	Przyjęta metodyka	6
3.3	Struktura podziału prac (WBS)	7
3.4	Estymacja pracochłonności (metoda trójpunktowa PERT)	7
3.5	Harmonogram — wykres Gantta	7
3.6	Kamienie milowe	9
3.7	Analiza ścieżki krytycznej (CPM)	9
3.8	Zarządzanie zasobami i obciążeniem	9
3.9	Zarządzanie ryzykiem i techniki zabezpieczające	10
3.10	Komunikacja i kontrola postępów	11
4	Analiza wymagań i orientacja na użytkownika	12
4.1	Wymagania funkcjonalne	12
4.2	Wymagania нефункционалне	13
4.3	Diagram przypadków użycia	13
4.4	Orientacja na użytkownika	13
5	Architektura i realizacja techniczna	13
5.1	Architektura systemu	14
5.2	Przepływ danych	14
5.3	Algorytm klasyfikacji nawierzchni	15
5.4	Klasyfikacja warunków pogodowych	15
5.5	Model danych	16
5.6	Konteneryzacja środowiska	16
6	Produkt końcowy i interfejs użytkownika	17
6.1	Wizualizacja na mapie	17
6.2	Prezentacja działającego interfejsu	17

6.3	Wybrane decyzje projektowe UX	19
7	Testowanie i weryfikacja	20
8	Podsumowanie i wnioski	20
8.1	Realizacja celów	20
8.2	Wnioski z pracy zespołowej	20
8.3	Kierunki rozwoju	21
	Bibliografia	21
	Spis rysunków	22
	Spis tablic	22

1. Wprowadzenie

1.1. Cel i geneza projektu

Stan nawierzchni dróg miejskich bezpośrednio wpływa na bezpieczeństwo uczestników ruchu, komfort jazdy oraz koszty eksploatacji pojazdów. Tradycyjna inwentaryzacja ubytków (dziur, wybojów, miejsc gwałtownego hamowania) prowadzona jest ręcznie, jest kosztowna i szybko traci aktualność. Celem projektu RoadVision było zaprojektowanie i zbudowanie działającego systemu, który automatycznie gromadzi dane z czujników pojazdu, w czasie rzeczywistym klasyfikuje stan nawierzchni i prezentuje wynik na interaktywnej mapie miasta.

System przygotowaliśmy jako rozwiązanie pilotażowe dla obszaru Rzeszowa, z możliwością łatwej rekonfiguracji na dowolny inny obszar — współrzędne mapy oraz źródło danych są zwykłymi parametrami konfiguracyjnymi.

1.2. Cel dokumentu

Sprawozdanie pełni dwie role. Po pierwsze, opisuje sposób, w jaki zorganizowaliśmy pracę dwuosobowego zespołu — przyjętą metodykę, podział prac (WBS), harmonogram, analizę ścieżki krytycznej, zarządzanie ryzykiem oraz kontrolę postępów. Po drugie, przedstawia rozwiązanie techniczne — architekturę, model danych, algorytmy oraz gotowy produkt. Zgodnie z charakterem przedmiotu Projekt Zespołowy najwięcej uwagi poświęciliśmy pierwszemu obszarowi.

1.3. Zakres i założenia

- Akwizycja danych z czterech źródeł: akcelerometr, GPS, czujnik temperatury/wilgotności oraz dane o zajętości parkingów.
- Klasyfikacja stanu nawierzchni do jednej z kategorii: **równa**, **wyboj**, **dziura**, **gwałtowne hamowanie**.
- Przetwarzanie strumieniowe w architekturze brzegowej (edge computing).
- Trwałe składowanie danych oraz dystrybucja w czasie rzeczywistym (WebSocket) do aplikacji wizualizującej.
- Pełna konteneryzacja środowiska uruchomieniowego (Docker Compose).

2. Zespół projektowy i organizacja pracy

2.1. Skład zespołu i podział obowiązków

Projekt realizowaliśmy we dwóch. Na początku podzieliliśmy się obowiązkami tak, aby każdy zajął się tym, co lubi i robi sprawniej, ale zostawiliśmy też kilka obszarów wspólnych — dzięki temu żaden moduł nie był znany tylko jednej osobie. Z grubsza podział wyglądał tak: Roman wziął na siebie część serwerową i infrastrukturę, a Rostysław stronę kliencką i prezentację danych. Staraliśmy się, żeby ilość pracy rozłożyła się po połowie.

Tabela 1. Podział obowiązków w zespole

Osoba	Czym się zajmował
Roman Bezsh-chasnyi	Architektura systemu, moduł Agent, klasyfikacja nawierzchni (Edge), buforowanie danych (Hub, Redis), konteneryzacja (Docker), prowadzenie repozytorium i harmonogramu.
Rostysław Yaremko	Moduł Store (API, baza danych, WebSocket), aplikacja MapView (mapa, trasa, markery), klasyfikacja pogody, testy scenariuszowe oraz dokumentacja wraz z materiałem graficznym.

Analizę wymagań i ogólny kształt architektury obmyśliliśmy razem — to były decyzje, które wpływały na pracę obu osób, więc nie chcieliśmy ich podejmować w pojedynkę. Część zadań przesuwaliśmy też między sobą w trakcie semestru, gdy ktoś miał akurat więcej czasu (więcej o tym w sekcji 3.8).

2.2. Macierz odpowiedzialności RACI

Żeby nie było wątpliwości, kto za co odpowiada, rozpisaliśmy zadania w prostej tabeli RACI (Responsible, Accountable, Consulted, Informed).

Tabela 2. Macierz RACI dla kluczowych pakietów prac

Pakiet pracy	Roman	Rostysław
Analiza wymagań i koncepcja systemu	A/R	C
Architektura i interfejsy modułów	A/R	C
Moduł Agent (akwizycja, MQTT)	A/R	I
Moduł Edge (klasyfikacja nawierzchni)	A/R	I
Moduł Hub (Redis, buforowanie)	A/R	I
Moduł Store (API, PostgreSQL, WebSocket)	C	A/R
Aplikacja MapView (Kivy)	I	A/R
Klasyfikacja warunków pogodowych	I	A/R
Konteneryzacja (Docker Compose)	A/R	I
Testy i scenariusze użycia	C	A/R
Dokumentacja i materiał graficzny	C	A/R

R – Wykonawca, A – Odpowiedzialny (rozliczany), C – Konsultowany, I – Informowany.

2.3. Narzędzia współpracy

Tabela 3. Narzędzia wykorzystane do organizacji pracy zespołowej

Obszar	Narzędzie / metoda
Kontrola wersji	Git + GitHub (gałęzie funkcyjne, Pull Requesty, code review)
Zarządzanie zadaniami	Tablica Kanban (backlog → w toku → przegląd → gotowe)
Komunikacja bieżąca	Komunikator zespołowy + krótkie synchronizacje (stand-up)
Środowisko uruchomieniowe	Docker Compose (powtarzalne środowisko dla obu osób)
Dokumentacja	L ^A T _E X (system kontroli wersji wspólny z kodem)

3. Zarządzanie projektem

3.1. Karta projektu

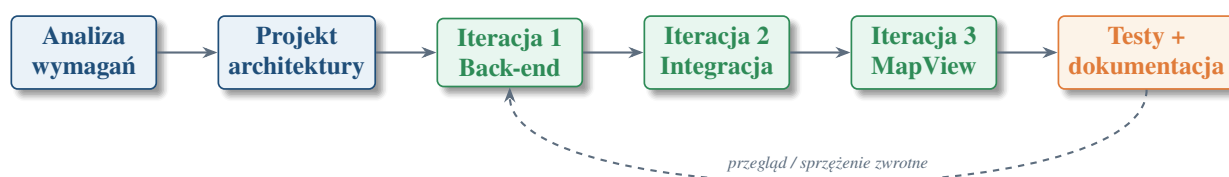
Zanim zabraliśmy się do kodu, spisaliśmy na jednej stronie najważniejsze ustalenia: po co robimy projekt, co dokładnie ma powstać, ile mamy czasu i kiedy uznamy, że się udało. Ta krótka notatka pomagała nam później wracać do tego, co naprawdę było ważne.

Tabela 4. Karta projektu RoadVision

Element	Opis
Cel	Zbudowanie działającego systemu monitorowania jakości nawierzchni z wizualizacją w czasie rzeczywistym.
Sponsor	Prowadzący przedmiot Projekt Zespołowy (WSliZ).
Zespół	2 osoby (kierownik/architekt, programista/wizualizacja).
Czas	05.05.2026 – 09.06.2026 (5 tygodni).
Ograniczenia	Środowisko studenckie, brak realnego sprzętu IoT (dane odtwarzane z CSV).
Kryteria sukcesu	Kompletny potok end-to-end, wizualizacja na mapie w czasie rzeczywistym, terminowe oddanie.
Poza zakresem	Produkcyjne wdrożenie, integracja z rzeczywistą flotą pojazdów.

3.2. Przyjęta metodyka

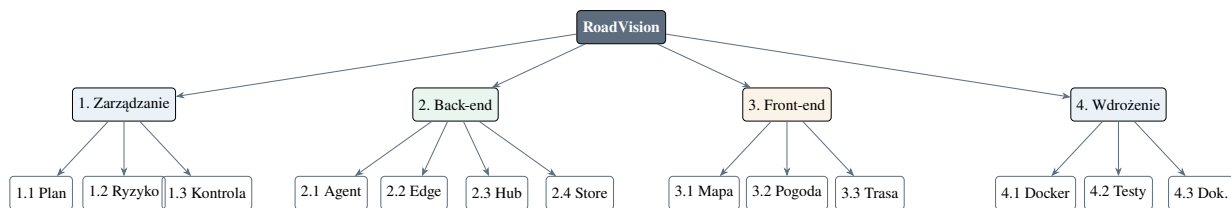
Na pierwszym spotkaniu zastanawialiśmy się, czy rozpisać cały projekt „z góry”, czy raczej pracować małymi krokami. Wyszło nam coś pośredniego. Rzeczy, które trudno później zmienić — analizę wymagań i ogólną architekturę — przemyśleliśmy dokładnie na samym początku. Resztę, czyli pisanie kolejnych modułów, podzieliliśmy na tygodniowe kawałki: pod koniec każdego tygodnia mieliśmy działający fragment i wspólnie go przeglądaliśmy. Dzięki temu nie musieliśmy nic dużego przerabiać, a problemy z łączeniem modułów wychodziły szybko.



Rysunek 1. Hybrydowy przepływ faz projektu — sekwencyjne planowanie i iteracyjna implementacja.

3.3. Struktura podziału prac (WBS)

Aby nie zgubić się w zakresie, rozpisaliśmy cały projekt na kartce, a następnie przenieśliśmy go do formy struktury podziału prac (WBS). Podzieliliśmy całość na cztery główne pakiety prac, a każdy z nich na drobniejsze zadania — dzięki temu łatwiej było nam oszacować pracochłonność i zbudować harmonogram. Struktura ta stała się dla nas punktem wyjścia przy podziale obowiązków między siebie.



Rysunek 2. Struktura podziału prac (Work Breakdown Structure) projektu RoadVision.

3.4. Estymacja pracochłonności (metoda trójpunktowa PERT)

Pracochłonność pakietów oszacowano metodą trójpunktową PERT, w której wartość oczekiwaną wyznacza się ze wzoru $t_e = \frac{o+4m+p}{6}$, gdzie o — szacunek optymistyczny, m — najbardziej prawdopodobny, p — pesymistyczny (w dniach roboczych). Podejście to uwzględnia niepewność i ogranicza ryzyko niedoszacowania (zabezpieczenie R5).

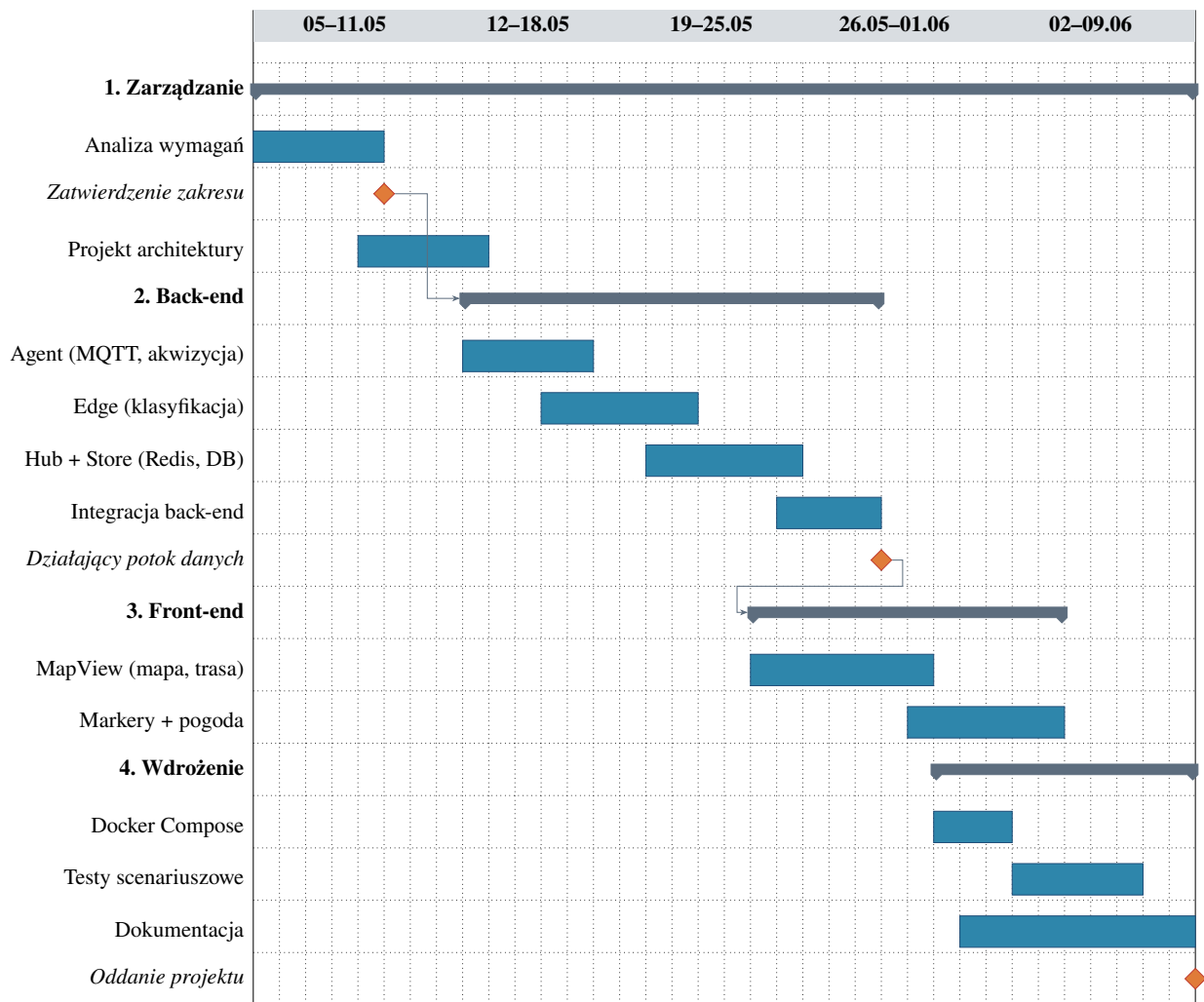
Tabela 5. Estymacja pracochłonności metodą PERT (w dniach roboczych)

Pakiet pracy	o	m	p	t_e
Analiza wymagań i architektura	4	6	9	6,2
Moduł Agent	2	3	5	3,2
Moduł Edge (klasyfikacja)	3	4	7	4,3
Hub + Store	3	5	8	5,2
Aplikacja MapView	5	7	11	7,3
Konteneryzacja i testy	3	4	6	4,2
Dokumentacja	3	5	8	5,2
Razem	23	34	54	35,6

3.5. Harmonogram — wykres Gantta

Realizację zaplanowano na okres od 5 maja do 9 czerwca 2026 (pięć tygodni roboczych). Harmonogram przedstawiono na wykresie Gantta (rys. 3); kamienie

milowe oznaczono rombami.



Rysunek 3. Harmonogram projektu RoadVision (05.05–09.06.2026) w formie wykresu Gantta.

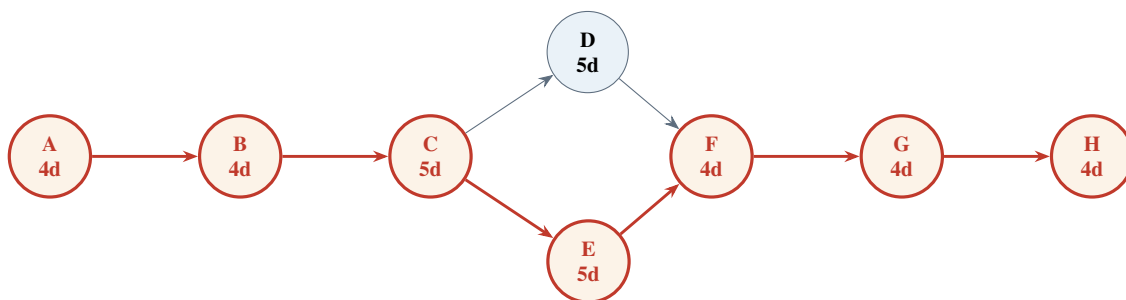
3.6. Kamienie milowe

Tabela 6. Kamienie milowe projektu

Data	Kamień milowy	Kryterium akceptacji
09.05.2026	Zatwierdzenie zakresu	Uzgodniona lista wymagań i kategorii stanu drogi.
13.05.2026	Zamrożenie architektury	Zdefiniowane interfejsy między modułami.
28.05.2026	Działający potok danych	Dane przechodzą Agent→Store bez błędów.
04.06.2026	Kompletny produkt (MVP)	Wizualizacja na mapie działa w czasie rzeczywistym.
09.06.2026	Oddanie projektu	Repozytorium i sprawozdanie gotowe do prezentacji.

3.7. Analiza ścieżki krytycznej (CPM)

W celu identyfikacji zadań decydujących o terminie zakończenia przeprowadzono analizę metodą ścieżki krytycznej (CPM). Diagram sieciowy (rys. 4) przedstawia zależności między czynnościami; ścieżkę krytyczną zaznaczono kolorem czerwonym. Każde opóźnienie zadania leżącego na tej ścieżce przekłada się bezpośrednio na opóźnienie całego projektu (zapas czasu = 0).



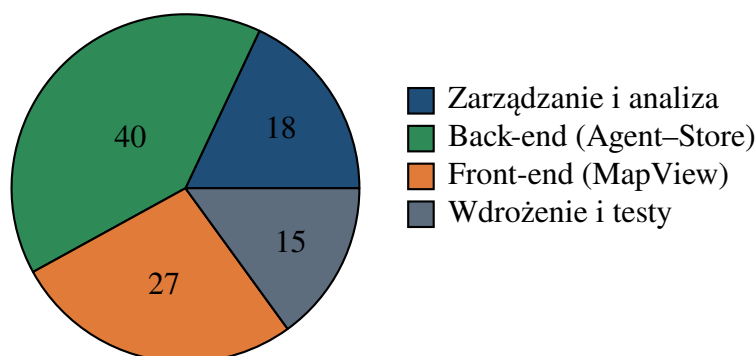
Rysunek 4. Diagram sieciowy CPM. Ścieżka krytyczna A–B–C–E–F–G–H (30 dni roboczych) wyróżniona kolorem czerwonym; czynność D (projekt MapView) posiada zapas czasu.

A – Analiza wymagań, B – Architektura, C – Edge (klasyfikacja), D – MapView, E – Hub/Store, F – Integracja, G – Konteneryzacja, H – Testy.

3.8. Zarządzanie zasobami i obciążeniem

Głównym ograniczonym zasobem w projekcie studenckim jest czas. Aby utrzymać równomierne obciążenie obu członków zespołu, pracochłonność esty-

mowano dla każdego pakietu pracy, a następnie bilansowano przydziały. Rozkład szacowanego nakładu pracy przedstawia rys. 5.



Rysunek 5. Szacunkowy rozkład nakładu pracy według pakietów WBS (%).

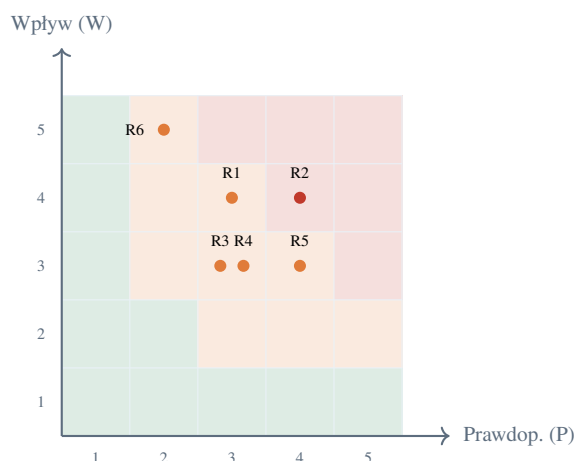
Staraliśmy się też nie obciążać obu osób naraz: zadania o niższym priorytecie przesuwaliśmy w obrębie ich zapasu czasu (por. CPM), żeby uniknąć tygodni, w których oba moduły wymagałyby intensywnej pracy w tym samym czasie. Podział obowiązków (opisany w sekcji 2.1) ustaliliśmy tak, aby każdy zajmował się zadaniami bliższymi jego umiejętnościom — dzięki temu praca szła szybciej.

3.9. Zarządzanie ryzykiem i techniki zabezpieczające

Na początku wspólnie wypisaliśmy wszystko, co — naszym zdaniem — mogło pójść nie tak, a następnie uporządkowaliśmy te zagrożenia w prostym rejestrze ryzyka. Każde z nich oceniliśmy pod kątem prawdopodobieństwa (P) i wpływu (W) w skali 1–5; iloczyn $P \times W$ wyznaczał priorytet reakcji. Dla ryzyk o najwyższym priorytecie z góry ustaliliśmy, co zrobimy — m.in. redundancję kompetencji, wczesną integrację oraz powtarzalne środowisko kontenerowe — tak, aby w razie problemu nie działać w pośpiechu.

Tabela 7. Rejestr ryzyka projektu (wybrane pozycje)

#	Ryzyko	P	W	$P \times W$	Reakcja / zabezpieczenie
R1	Niedostępność jednego z członków zespołu	3	4	12	Redundancja wiedzy (overlap ról), code review, dokumentacja na bieżąco.
R2	Problemy z integracją modułów (MQT-T/HTTP/WS)	4	4	16	Wczesna integracja „end-to-end”, wspólne kontrakty danych (Pydantic).
R3	Rozbieżność środowisk uruchomieniowych	3	3	9	Konteneryzacja Docker Compose — identyczne środowisko dla obu osób.
R4	Błędna kalibracja progów klasyfikacji	3	3	9	Parametryzacja progów, testy na danych odtwarzanych z plików CSV.
R5	Niedoszacowanie czasu na dokumentację	4	3	12	Rozpoczęcie dokumentacji równolegle z implementacją (od 21.05).
R6	Utrata danych / brak kontroli wersji	2	5	10	Repozytorium Git, częste commity, gałęzie funkcyjne.



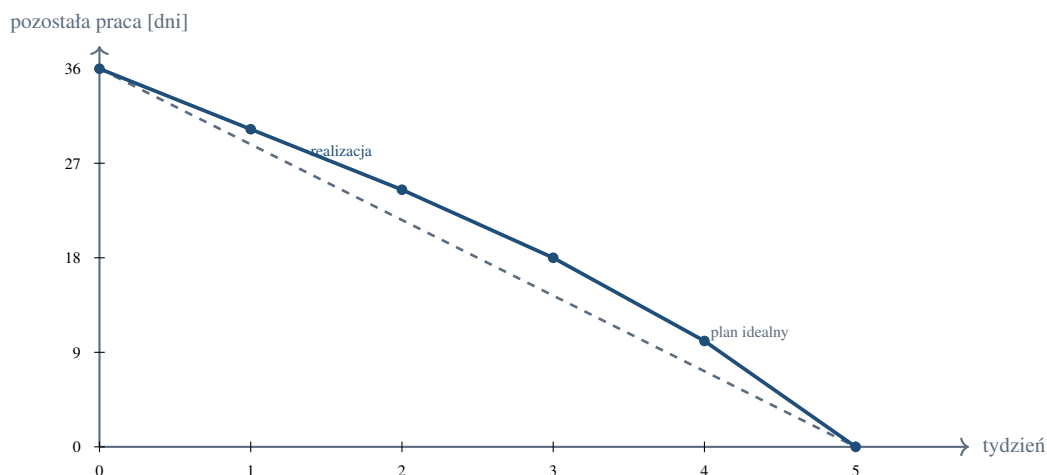
Rysunek 6. Macierz ryzyka — pozycjonowanie zagrożeń względem prawdopodobieństwa i wpływu (kolor strefy odpowiada iloczynowi $P \times W$). W strefie krytycznej znalazło się wyłącznie ryzyko R2.

Rozkład zagrożeń odzwierciedla specyfikę dwuosobowego zespołu: większość ryzyk ma podwyższony wpływ, ponieważ każdy moduł i każda osoba są dla projektu kluczowe (brak rezerwy kadrowej). Mimo to tylko jedno ryzyko — trudności z integracją modułów (R2) — trafiło do strefy krytycznej; pozostałe, dzięki zaplanowanym z góry zabezpieczeniom, utrzymały się w strefie umiarkowanej.

3.10. Komunikacja i kontrola postępów

Komunikację w zespole oparto na krótkich, regularnych synchronizacjach oraz asynchronicznej wymianie informacji przez komunikator i komentarze w Pull Requestach. Postęp kontrolowano w cyklu tygodniowym, porównując stan faktyczny z planem (kamienie milowe). Pomagał w tym wzajemny przegląd kodu — żadna zmiana nie trafiała do gałęzi głównej bez akceptacji drugiej osoby, co przy okazji realizowało zabezpieczenie R1 (obie osoby orientowały się w całości projektu).

Postęp śledziliśmy też prostym wykresem wypalania (burndown), porównującym pozostałą pracę z planem. Rysunek 7 pokazuje, że tempo prac było zbliżone do planu, z niewielkim przyspieszeniem w ostatnim tygodniu.



Rysunek 7. Wykres wypalania (burndown) — pozostała pracochłonność w kolejnych tygodniach.

Tabela 8. Plan komunikacji zespołowej

Zdarzenie	Częstotliwość	Cel
Synchronizacja (stand-up)	2–3 razy w tygodniu	Status, blokery, plan na kolejne dni.
Przegląd kodu (PR review)	Przy każdej zmianie	Kontrola jakości, transfer wiedzy.
Przegląd iteracji	Co tydzień	Ocena przyrostu względem kamieni milowych.
Retrospektywa	Na koniec projektu	Wnioski na przyszłość.

4. Analiza wymagań i orientacja na użytkownika

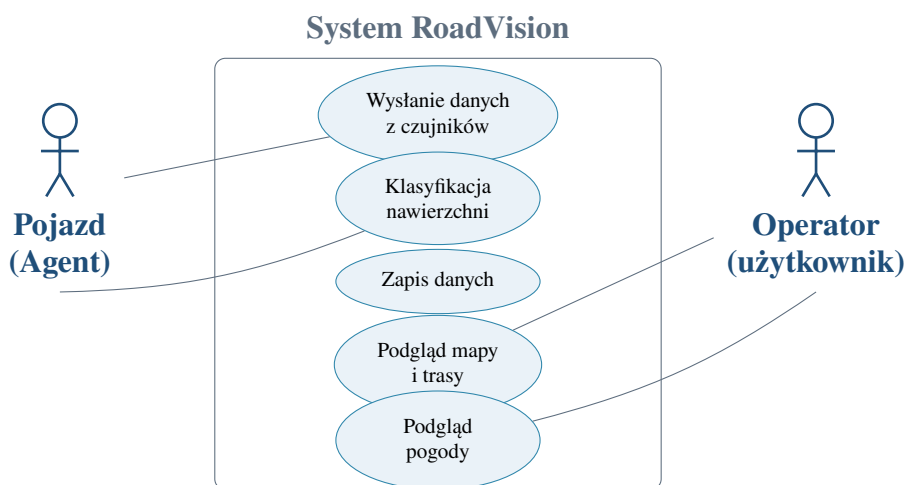
4.1. Wymagania funkcjonalne

- F1.** System akwiruje dane z czujników pojazdu (akcelerometr, GPS, warunki).
- F2.** System klasyfikuje stan nawierzchni do jednej z czterech kategorii.
- F3.** System trwale zapisuje przetworzone dane w bazie danych.
- F4.** System udostępnia dane w czasie rzeczywistym przez WebSocket.
- F5.** Aplikacja kliencka prezentuje pozycję pojazdu i trasę na mapie.
- F6.** Aplikacja oznacza wykryte ubytki na mapie ikonami (bez duplikatów).
- F7.** Aplikacja prezentuje aktualne warunki pogodowe wraz z ikoną.

4.2. Wymagania niefunkcjonalne

- N1.** Wydajność: przetwarzanie strumieniowe z opóźnieniem rzędu pojedynczych sekund.
- N2.** Skalowalność: buforowanie i grupowanie zapisów (Redis, batch).
- N3.** Przenośność: pełna konteneryzacja, uruchomienie jednym poleceniem.
- N4.** Łatwość konfiguracji: obszar mapy i parametry jako zmienne środowiskowe (np. zmiana z Kijowa na Rzeszów).
- N5.** Utrzymywalność: architektura warstwowa (porty i adaptery).

4.3. Diagram przypadków użycia



Rysunek 8. Diagram przypadków użycia systemu RoadVision.

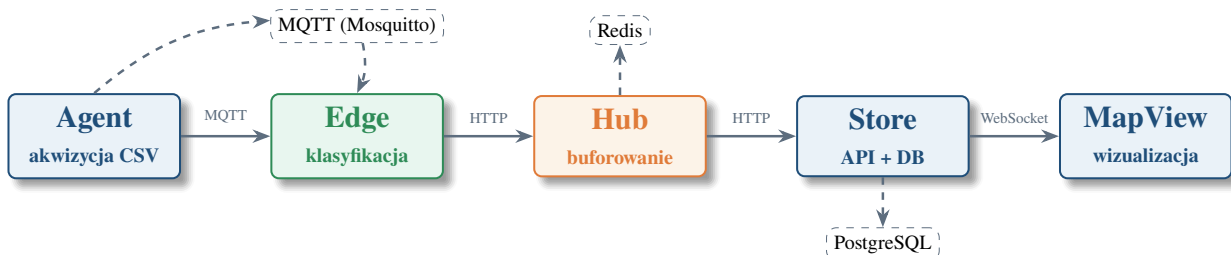
4.4. Orientacja na użytkownika

Produkt zaprojektowano z myślą o końcowym odbiorcy — operatorze służb drogowych lub kierowcy. Decyzje projektowe podporządkowano czytelności: stan nawierzchni kodowany jest kolorem i ikoną, a nie wartością liczbową; trasa pojazdu jest rysowana na bieżąco; warunki pogodowe prezentowane są piktogramem zrozumiałym bez czytania. Zastosowano też mechanizm eliminacji duplikatów markerów (siatka przestrzenna), aby mapa pozostała czytelna nawet przy wielokrotnym przejeździe tą samą ulicą.

5. Architektura i realizacja techniczna

5.1. Architektura systemu

RoadVision zbudowano w architekturze mikroserwisowej, w której każdy komponent realizuje pojedynczą odpowiedzialność i komunikuje się z pozostałymi przez jasno zdefiniowane interfejsy. Wewnętrznie moduły back-endowe zorganizowano zgodnie ze wzorcem portów i adapterów (architektura heksagonalna): logika domenowa (usecases, entities) jest niezależna od szczegółów technologii (adapters).



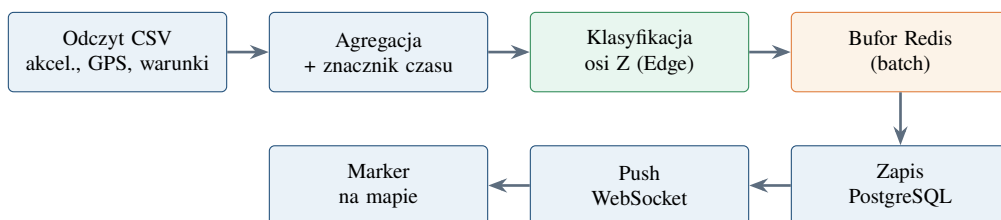
Rysunek 9. Architektura mikroserwisowa RoadVision wraz z infrastrukturą pośredniczącą.

Tabela 9. Komponenty systemu, odpowiedzialności i wykonawca

Moduł	Technologie	Odpowiedzialność	Wykonawca
Agent	Python, paho-mqtt	Odczyt danych z czujników (CSV), publikacja przez MQTT.	Roman
Edge	Python, paho-mqtt, Pydantic	Klasyfikacja stanu nawierzchni, przekazanie do Hub (HTTP).	Roman
Hub	FastAPI, Redis	Buforowanie i grupowanie (batch) rekordów, przekazanie do Store.	Roman
Store	FastAPI, SQLAlchemy, PostgreSQL	Trwały zapis (CRUD) i rozsył w czasie rzeczywistym (WebSocket).	Rostysław
MapView	Kivy, kivy-garden.mapview	Wizualizacja pozycji, trasy, ubytków i pogody na mapie.	Rostysław

5.2. Przepływ danych

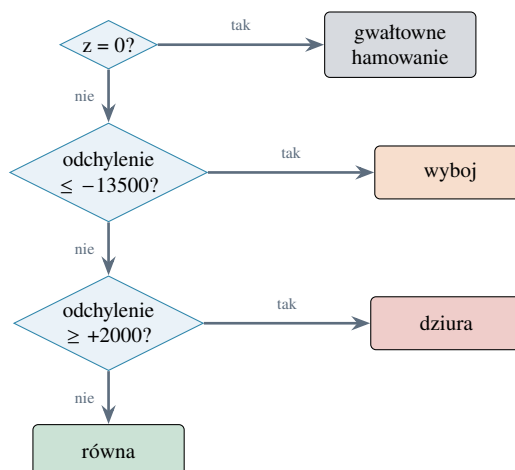
Dane przepływają jednokierunkowo od pojazdu do interfejsu użytkownika. Rysunek 10 przedstawia kolejne przekształcenia surowego odczytu czujnika w oznaczenie na mapie.



Rysunek 10. Potok przetwarzania danych — od odczytu czujnika do markera na mapie.

5.3. Algorytm klasyfikacji nawierzchni

Sercem warstwy brzegowej jest reguła klasyfikująca stan drogi na podstawie odczytu osi Z akcelerometru. Wartość spoczynkowa (NORMAL_Z) wynosi 16384; odchylenie od niej interpretowane jest jako wyboj, dziura lub odcinek równy. Wartość zerowa traktowana jest jako gwałtowne zatrzymanie pojazdu. Moduł Edge, wraz z poprzedzającym go modułem Agent, przygotował Roman.



Rysunek 11. Drzewo decyzyjne klasyfikacji stanu nawierzchni w module Edge.

```

1 z = agent_data.accelerometer.z
2 if z == 0:
3     return ProcessedAgentData(road_state="sudden_stop", agent_data=agent_data)
4
5 NORMAL_Z = 16384
6 deviation = z - NORMAL_Z
7 lower_limit, upper_limit = -13500, 2000
8
9 if lower_limit < deviation < upper_limit:
10     road_state = "smooth"
11 elif deviation <= lower_limit:
12     road_state = "bump"
13 else:
14     road_state = "pothole"
  
```

Listing 1. Reguła klasyfikacji nawierzchni (edge/app/usecases/data_processing.py).

5.4. Klasyfikacja warunków pogodowych

Niezależnie od stanu drogi system klasyfikuje warunki atmosferyczne na podstawie temperatury, wilgotności oraz flagi czujnika oblodzenia (ESP). Wynik prezentowany jest piktogramem w lewym górnym rogu mapy. Tę część — wraz z aplikacją MapView — zrealizował Rostysław.

Tabela 10. Reguły klasyfikacji warunków pogodowych

Warunek	Kryterium
Śliska droga (ESP)	flaga ESP = prawda
Oblodzenie / mgła lodowa	wilgotność > 85% i temperatura < 2°C
Mgła	wilgotność > 85% i temperatura < 10°C
Deszcz	wilgotność > 85%
Pochmurno	wilgotność > 70%
Częściowe zachmurzenie	wilgotność > 55%
Słonecznie	pozostałe przypadki

5.5. Model danych

Przetworzone rekordy zapisywane są w pojedynczej tabeli relacyjnej `processed_agent_data`, łączącej stan drogi z odczytami czujników i pozycją GPS. Warstwę zapisu i udostępniania danych (moduł Store) przygotował Rostysław, natomiast pośredniczący moduł Hub — Roman.

<code>processed_agent_data</code>
<code>id : SERIAL (PK)</code>
<code>road_state : VARCHAR</code>
<code>user_id : INTEGER</code>
<code>x, y, z : FLOAT (akcelerometr)</code>
<code>latitude, longitude : FLOAT (GPS)</code>
<code>temperature, humidity : FLOAT</code>
<code>esp : BOOL</code>
<code>timestamp : TIMESTAMP</code>

Rysunek 12. Schemat tabeli `processed_agent_data` w bazie PostgreSQL.

5.6. Konteneryzacja środowiska

Całe środowisko uruchomieniowe opisano w pliku `docker-compose.yaml`. Pojedyncze polecenie uruchamia osiem powiązanych usług, izolowanych w dedykowanych sieciach. Zapewnia to powtarzalność (realizacja zabezpieczenia R3) i ułatwia wdrożenie. Konfigurację konteneryzacji oraz wspólne repozytorium prowadził Roman.

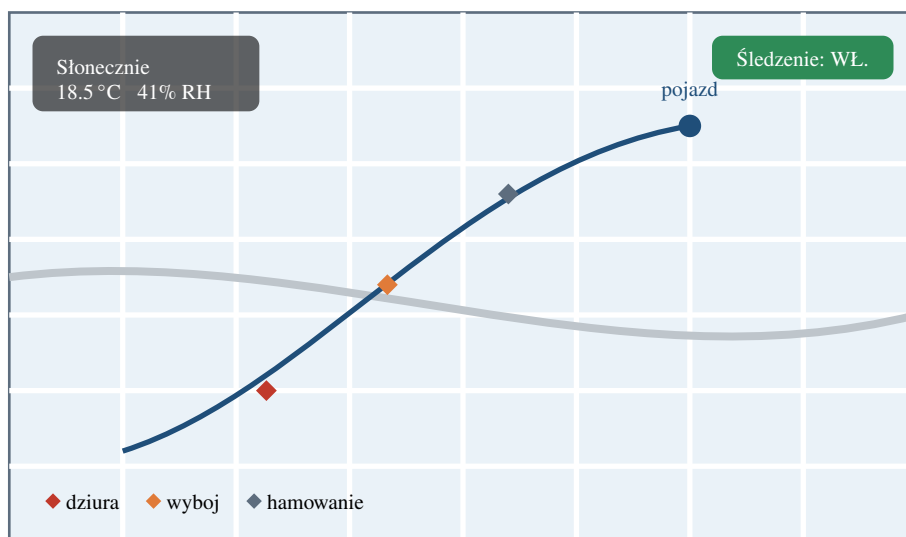
Tabela 11. Usługi zdefiniowane w Docker Compose

Usługa	Obraz	Rola
mqtt	eclipse-mosquitto	Broker komunikatów MQTT.
agent	build ../agent	Źródło danych z czujników.
edge	build ../edge	Klasyfikacja brzegowa.
redis	redis	Bufor kolejkowy dla Hub.
hub	build ../hub	Grupowanie i przekazywanie danych.
store	build ../store	API, baza danych, WebSocket.
postgres_db	postgres:16	Trwałe składowanie danych.
pgadmin	dpage/pgadmin4	Administracja bazą danych.

6. Produkt końcowy i interfejs użytkownika

6.1. Wizualizacja na mapie

Aplikacja MapView, napisana w bibliotece Kivy, stanowi twarz produktu. Po nawiązaniu połączenia WebSocket prezentuje na mapie aktualną pozycję pojazdu (ikona samochodu), rysuje przebytą trasę oraz nanosi ikony wykrytych ubytków. Obszar wyświetlanej mapy jest parametrem konfiguracyjnym (zob. wymaganie N4). Rysunek 13 przedstawia szkic interfejsu (do zastąpienia zrzutem ekranu działającej aplikacji).

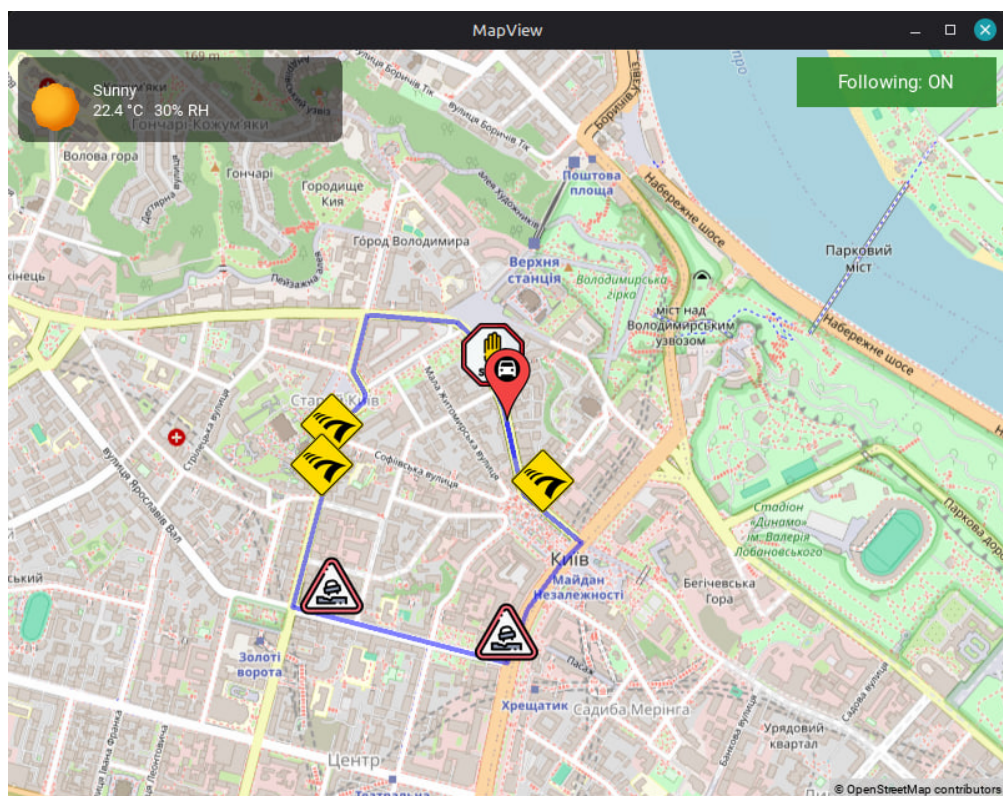


Rysunek 13. Szkic interfejsu aplikacji MapView (mapa miasta, trasa, markery ubytków, widget pogody).

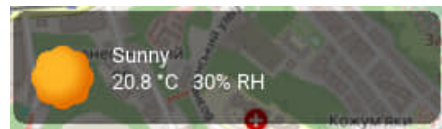
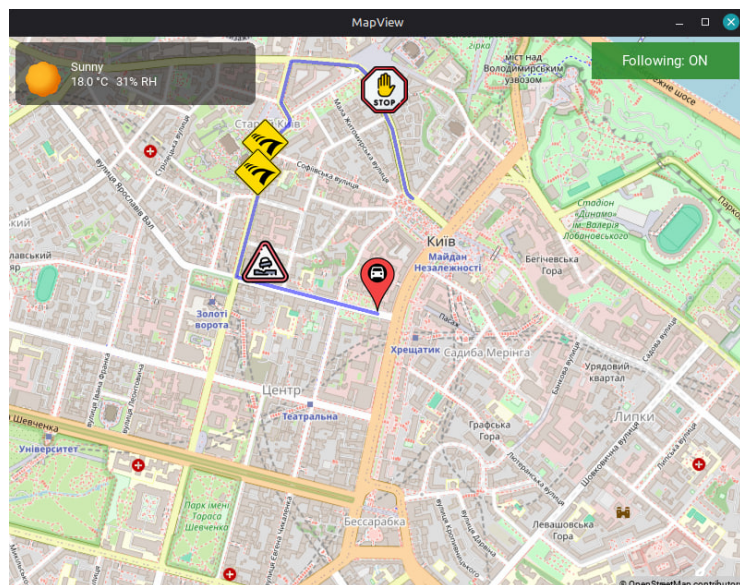
6.2. Prezentacja działającego interfejsu

Poniżej zamieszczono zrzuty ekranu z działającej aplikacji MapView, ilustrujące kluczowe funkcje produktu w trakcie testowego przejazdu.

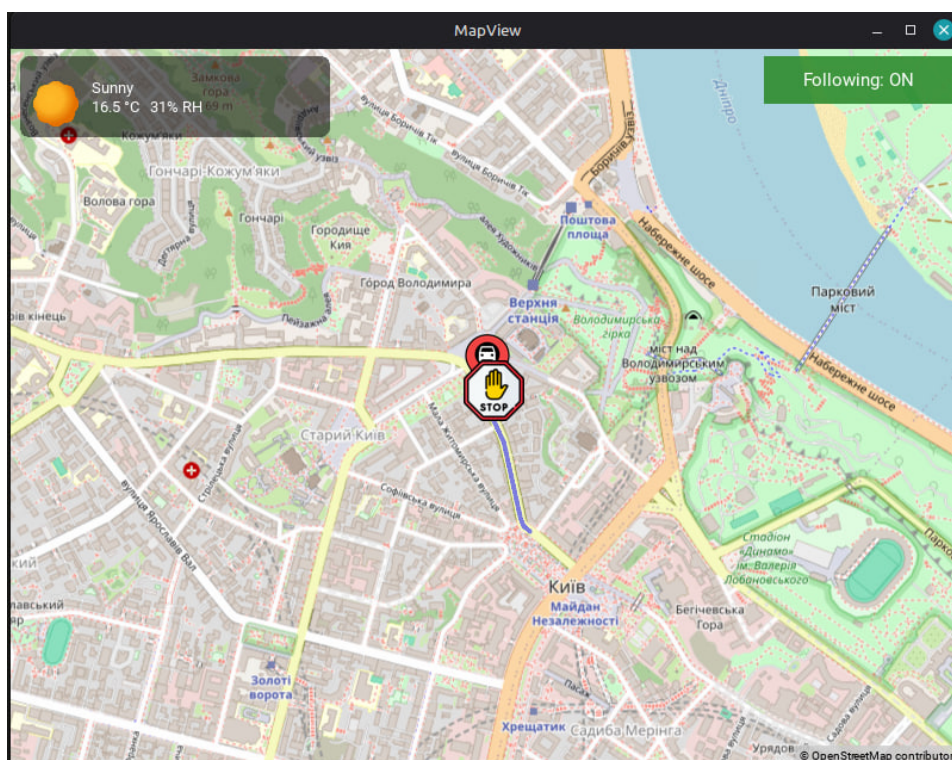
Jako obszar testowy do prezentacji wybrano Kijów, a nie docelowy Rzeszów. Zdecydowały o tym dwa względy praktyczne. Po pierwsze, gęstsza i bardziej zróżnicowana sieć drogowa Kijowa dostarcza znacznie więcej miejsc problematycznych (ubytków i nierówności), dzięki czemu działanie klasyfikatora można pokazać pełniej i na większej liczbie przypadków. Po drugie, dla tego obszaru dysponowaliśmy bogatszym zbiorem danych referencyjnych oraz narzędzi wspierających rozpoznawanie defektów; odpowiedników dopasowanych do potrzeb projektu dla Rzeszowa nie udało się pozyskać. Sam obszar mapy pozostaje przy tym parametrem konfiguracyjnym (wymaganie N4), więc przełączenie wizualizacji na Rzeszów sprowadza się do zmiany współrzędnych środka mapy.



Rysunek 14. Główny ekran aplikacji MapView: pozycja pojazdu, przebyta trasa oraz markery wykrytych defektów (dziury, wyboje, gwałtowne hamowanie).



Rysunek 15. Markery defektów na trasie — znak STOP (gwałtowne hamowanie), wyboje i dziury (po lewej) oraz widget warunków pogodowych z piktogramem i odczytami temperatury i wilgotności (po prawej).



Rysunek 16. Tryb śledzenia pojazdu (Following: ON) — mapa automatycznie centruje się na bieżącej pozycji pojazdu.

6.3. Wybrane decyzje projektowe UX

- Siatka przestrzenna (komórki ok. 30×30 m) zapobiega wielokrotnemu oznaczaniu tego samego ubytku przy ponownym przejeździe.

- Tryb śledzenia pojazdu można włączać/wyłączać jednym przyciskiem.
- Automatyczny reset trasy przy „przeskoku” GPS powyżej 500 m zapobiega rysowaniu nieprawdziwych, długich odcinków.

7. Testowanie i weryfikacja

Weryfikację prowadzono na danych odtwarzanych z plików CSV, co zapewniło powtarzalność scenariuszy. Testy obejmowały zarówno poprawność klasyfikacji, jak i działanie całego potoku „od końca do końca”.

Tabela 12. Wybrane scenariusze testowe

#	Scenariusz	Oczekiwany wynik	Status
T1	Odczyt akcelerometru w normie	Stan „równa”	OK
T2	Duże ujemne odchylenie osi Z	Stan „wyboj”	OK
T3	Dodatnie odchylenie osi Z	Stan „dziura”	OK
T4	Odczyt $z = 0$	„gwałtowne hamowanie”	OK
T5	Pełny potok Agent→MapView	Marker na mapie	OK
T6	Ponowny przejazd tą samą ulicą	Brak duplikatu markera	OK

8. Podsumowanie i wnioski

8.1. Realizacja celów

Wszystkie założone wymagania funkcjonalne zostały zrealizowane: powstał kompletny, działający potok przetwarzania danych — od akwizycji w pojeździe, przez klasyfikację brzegową i trwały zapis, po wizualizację w czasie rzeczywistym na mapie miasta. Projekt ukończono w zaplanowanym oknie czasowym (05.05–09.06.2026).

8.2. Wnioski z pracy zespołowej

Najważniejszym wnioskiem jest skuteczność wczesnej integracji oraz redundancji kompetencji jako technik zabezpieczających. Dzięki zbudowaniu szkieletu „end-to-end” na wczesnym etapie zespół wykrył problemy interfejsów

(najwyżej ocenione ryzyko R2) zanim stały się kosztowne. Konteneryzacja wyeliminowała klasę błędów „u mnie działa”. Elastyczny podział obowiązków pomiędzy przedmiotami pozwolił efektywnie wykorzystać mocne strony obu członków zespołu. Zastosowane techniki — WBS, wykres Gantta, analiza ścieżki krytycznej, rejestr ryzyka i macierz RACI — okazały się adekwatne nawet dla małego zespołu i znacząco zwiększyły przewidywalność prac.

8.3. Kierunki rozwoju

- Zastąpienie odtwarzania z CSV strumieniem z rzeczywistego urządzenia IoT.
- Uczenie maszynowe do klasyfikacji nawierzchni zamiast progów stałych.
- Agregacja danych od wielu pojazdów (mapa zbiorcza dla miasta).
- Wersja webowa lub mobilna aplikacji MapView.

Literatura

- [1] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, wyd. 6, PMI, 2017.
- [2] H. Kerzner, *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*, Wiley, 2017.
- [3] S. Ramírez, *FastAPI — dokumentacja oficjalna*. <https://fastapi.tiangolo.com>
- [4] OASIS, *MQTT Version 3.1.1 — specyfikacja protokołu*, 2014.
- [5] *Redis — dokumentacja*. <https://redis.io/docs>
- [6] *PostgreSQL 16 — dokumentacja*. <https://www.postgresql.org/docs/16/>
- [7] *Kivy — Cross-platform Python Framework*. <https://kivy.org>
- [8] *Docker Compose — dokumentacja*. <https://docs.docker.com/compose/>
- [9] A. Cockburn, *Hexagonal Architecture (Ports and Adapters)*, 2005.

Spis rysunków

1	Hybrydowy przepływ faz projektu — sekwencyjne planowanie i iteracyjna implementacja.	6
2	Struktura podziału prac (Work Breakdown Structure) projektu RoadVision.	7
3	Harmonogram projektu RoadVision (05.05–09.06.2026) w formie wykresu Gantta.	8
4	Diagram sieciowy CPM. Ścieżka krytyczna A–B–C–E–F–G–H (30 dni roboczych) wyróżniona kolorem czerwonym; czynność D (projekt MapView) posiada zapas czasu.	9
5	Szacunkowy rozkład nakładu pracy według pakietów WBS (%).	10
6	Macierz ryzyka — pozycjonowanie zagrożeń względem prawdopodobieństwa i wpływu (kolor strefy odpowiada iloczynowi $P \times W$). W strefie krytycznej znalazło się wyłącznie ryzyko R2.	11
7	Wykres wypalania (burndown) — pozostała pracochłonność w kolejnych tygodniach.	12
8	Diagram przypadków użycia systemu RoadVision.	13
9	Architektura mikroserwisowa RoadVision wraz z infrastrukturą pośredniczącą.	14
10	Potok przetwarzania danych — od odczytu czujnika do markera na mapie.	14
11	Drzewo decyzyjne klasyfikacji stanu nawierzchni w module Edge.	15
12	Schemat tabeli <code>processed_agent_data</code> w bazie PostgreSQL.	16
13	Szkic interfejsu aplikacji MapView (mapa miasta, trasa, markery ubytków, widget pogody).	17
14	Główny ekran aplikacji MapView: pozycja pojazdu, przebyta trasa oraz markery wykrytych defektów (dziury, wyboje, gwałtowne hamowanie).	18
15	Markery defektów na trasie — znak STOP (gwałtowne hamowanie), wyboje i dziury (po lewej) oraz widget warunków pogodowych z piktogramem i odczytami temperatury i wilgotności (po prawej).	19
16	Tryb śledzenia pojazdu (Following: ON) — mapa automatycznie centruje się na bieżącej pozycji pojazdu.	19

Spis tabel

1	Podział obowiązków w zespole	4
2	Macierz RACI dla kluczowych pakietów prac	5
3	Narzędzia wykorzystane do organizacji pracy zespołowej	5

4	Karta projektu RoadVision	6
5	Estymacja pracochłonności metodą PERT (w dniach roboczych)	7
6	Kamienie milowe projektu	9
7	Rejestr ryzyka projektu (wybrane pozycje)	10
8	Plan komunikacji zespołowej	12
9	Komponenty systemu, odpowiedzialności i wykonawca	14
10	Reguły klasyfikacji warunków pogodowych	16
11	Usługi zdefiniowane w Docker Compose	17
12	Wybrane scenariusze testowe	20